

ORM 框架和 MyBatis 详解—复习要点

1 ORM 框架概述

1.1 什么是 ORM (Object-Relation Mapping, 对象关系映射) [p.4]

- > **持久化 (Persistence):** 把瞬态数据 (如内存中的对象) 保存到可永久保存的存储设备 (磁盘文件、数据库等) 中 [p.4]
- > **持久化目标的三类** [p.4]:
 - 无结构文本文件: 通过 I/O 技术读写文件
 - 结构化的文本文件: 通过 SDK 提供的 API 读写文件
 - 关系型数据库: 通过数据库驱动技术 (如 JDBC) 读写 DBMS
- > **ORM 定义:** 完成瞬态的对象数据到持久的关系型数据映射的机制, 简称 ORM [p.4]

1.2 为什么需要 ORM 框架? [p.5]

- > **痛点:** Java 应用中, 数据库表结构与 Java 对象结构不一致, 开发者需编写大量重复的 JDBC 代码 (执行 SQL、设置参数、处理结果集、管理连接), 繁琐且易出错 [p.5]
- > **ORM 的四大目标** [p.5]:
 - 简化 JDBC 操作: 封装重复的连接管理和异常处理逻辑, 提供更友好的 API
 - 提高生产力: 将关系数据以对象的形式表示, 便于开发者理解与操作
 - 跨数据库移植性: 通过方言 (Dialect) 抽象提供一定程度的数据库无关性
 - 提供高级功能: 缓存、延迟加载 (Lazy Loading)、事务管理、批量操作等

1.3 回顾 JDBC API 编程 [p.6, p.9–11]

- > JDBC (Java Database Connectivity) 是 Java 标准库提供的 API, 用于执行 SQL 语句并处理结果集 [p.6]
- > **核心接口:** Driver、Connection、Statement、PreparedStatement、ResultSet [p.6]
- > **JDBC 编程痛点** [p.6]:
 - 大量样板代码 (Boilerplate Code): 获取连接、创建 PreparedStatement、设置参数、遍历 ResultSet、释放资源
 - 代码冗长, 容易出错
 - 忘记释放资源可能导致连接泄漏 (Connection Leak)
- > **传统 JDBC 编程流程** (以 userinfo 表为例) [p.9–11]:
 - DBUtil 工具类: 封装 getConnection()、closeConnection()、closeStatement()、closePreparedStatement()、closeResultSet() [p.9]
 - INSERT 操作: 获取连接 → 预编译 SQL → setXxx() 设置参数 → executeUpdate() → 关闭资源 [p.10]
 - SELECT 操作: 获取连接 → 执行查询 → while 循环遍历 ResultSet → 手动逐字段 setter 映射到 Java 对象 → 关闭资源 [p.11]

1.4 ORM 的实现方式（四种层次）

[p.7]

- > 裸 JDBC：最底层，完全手动控制，代码量大且易出错 [p.7]
- > 简化的 JDBC：如 Spring JDBC Template，封装连接管理和异常处理 [p.7]
- > 半自动 ORM 框架：如 MyBatis，开发者编写 SQL，框架负责参数/结果映射 [p.7]
- > 全自动 ORM 框架：如 Hibernate、Spring Data JPA，框架自动生成 SQL [p.7]

2 Spring JDBC 模板 (JdbcTemplate)

2.1 JdbcTemplate 简介

[p.13–14]

- > JdbcTemplate 是 Spring 框架在 JDBC 层提供的中心委托类 (Central Delegate Class) [p.13]
- > 核心能力 [p.13]:
 - 封装典型的 JDBC 流程：获取连接、创建语句、执行 SQL、遍历 ResultSet、处理异常、关闭资源
 - 让开发者只需专注于 SQL 和结果映射
 - 回调接口 (Callback Interfaces)：PreparedStatementCreator (SQL 创建)、ResultSetExtractor (结果提取)、RowMapper (行映射)
 - 统一异常处理：将 SQLException 转换为 Spring 的 DataAccessException 层次结构 (非受检异常)
 - 线程安全 (Thread-safe)：配置完成后可在多线程场景安全共享
 - 日志记录：所有 SQL 操作在 DEBUG 级别记录，便于排查问题
- > 架构角色：在程序员代码与 JDBC API 之间提供统一的模板方法，在保留代码灵活性的基础上尽量减少持久化代码 [p.14]

2.2 使用步骤

[p.15–16]

- > 引入依赖 (spring-boot-starter-jdbc + 数据库驱动) [p.15]
- > 配置数据源 (DataSource) [p.15]
- > 注入 JdbcTemplate 对象, 直接调用 API: query()、queryForObject()、queryForList()、update()、batchUpdate() 等 [p.16]

2.3 JdbcTemplate 的优缺点

[p.17]

- > 优点：简单、轻量，适用于小型项目、简单查询、对 SQL 有完全控制需求的场景 [p.17]
- > 缺点 [p.17]:
 - SQL 仍需手动编写，且在 Java 代码中拼写 (无 XML/注解分离，维护不便)
 - 结果映射需要手动指定 RowMapper (字段多时代码量大)
 - 没有缓存机制
 - 关联查询复杂时，映射麻烦
- > 适用边界：当查询复杂、需要动态拼接语句或关联关系映射时，需要更高级的 ORM 框架 [p.17]

3 MyBatis 框架

3.1 MyBatis 简介与历史

[p.19–20]

- > 原名 iBatis，由 Clinton Begin 于 2002 年发布 [p.19]
- > 2010 年项目被贡献给 Google Code，改名为 MyBatis [p.19]

- > 官网: <https://mybatis.org/mybatis-3/> [p.20]
- > **定位:** 一款优秀的持久层框架 (Persistence Framework), 支持自定义 SQL、存储过程 (Stored Procedure) 以及高级映射 (Advanced Mapping) [p.19]
- > 免除了几乎所有的 JDBC 代码以及设置参数和获取结果集的工作 [p.19]
- > 可通过简单的 XML 或注解 (Annotation) 来配置和映射原始类型、接口和 Java POJO (Plain Old Java Objects) 为数据库中的记录 [p.19]

3.2 MyBatis 核心特点 [p.22]

- > **(1) 自定义 SQL:** 开发者编写完整的 SQL 语句, 框架不会生成或修改 SQL, 保证精确控制 [p.22]
- > **(2) 参数映射与结果映射:** 将参数对象的属性映射为预编译语句参数 (`#{} 占位符`), 根据配置将结果行映射为 Java 对象 [p.22]
- > **(3) 动态 SQL:** 提供 `<if>`、`<choose>`、`<when>`、`<otherwise>`、`<foreach>`、`<where>`、`<set>`、`<trim>` 等标签在 XML 中拼接条件, 避免 Java 代码中字符串拼接 [p.22]
- > **(4) 映射文件简洁:** 一个 Mapper XML 文件包含 `<select>`、`<insert>`、`<update>`、`<delete>` 基本元素以及 `<resultMap>` 定义 [p.22]
- > **(5) 缓存机制 [p.22]:**
 - 一级缓存 (Local Cache): SqlSession 级别, 默认开启, 同一 SqlSession 内相同查询走缓存
 - 二级缓存 (Second-level Cache): Mapper 级别, 可选开启, 跨 SqlSession 共享, 支持自定义缓存实现 (如 Ehcache、Redis)
- > **(6) 插件机制 (Plugin/Interceptor):** 允许编写拦截器在语句执行前后插入逻辑 (基于动态代理 + 责任链模式), 典型应用: 分页插件、性能分析、SQL 阻断 [p.22]
- > **(7) 与 Spring 集成良好:** 通过 MyBatis-Spring Boot Starter, 自动扫描 Mapper 接口并创建代理对象, 与 Spring 事务管理完美结合 [p.22]

3.3 MyBatis 基本使用流程 [p.23–26]

Step 1. 引入依赖: mybatis-spring-boot-starter + 数据库驱动 [p.23]

Step 2. 配置框架: application.yml / application.properties 中配置数据源、Mapper 扫描路径等 [p.23]

Step 3. 创建实体类 (Entity / POJO): 属性与数据库表字段对应 [p.24]

Step 4. 创建 Mapper 接口: 定义数据访问方法 (接口 + 方法签名), 无需实现类 [p.24]

Step 5. 编写 Mapper XML 映射文件 [p.25]:

- > `<mapper namespace="...">`: namespace 对应 Mapper 接口全限定名
- > SQL 元素: `<select>`、`<insert>`、`<update>`、`<delete>`
- > id 对应接口方法名, parameterType 指定入参类型, resultType / resultMap 指定返回值类型
- > `#{} 占位符` 用于预编译参数绑定, 防止 SQL 注入

Step 6. 在 Service 层注入并使用 Mapper: 通过 `@Autowired` 注入 Mapper 接口代理对象 [p.26]

3.4 动态 SQL 详解 [p.22, p.27]

- > **目的:** 根据条件动态拼接 SQL, 避免 Java 代码中大量 if-else 字符串拼接 [p.22, p.27]
- > **常用动态 SQL 标签:**
 - `<if test="...">`: 条件判断, 根据布尔表达式决定是否包含片段
 - `<choose>` / `<when>` / `<otherwise>`: 多条件选择, 类似 Java switch-case

- `<foreach collection="..." item="..." open="(" close=")" separator=",">`: 遍历集合, 常用于 IN 查询
- `<where>`: 自动处理 WHERE 子句前缀, 智能去除多余的 AND/OR
- `<set>`: 动态 UPDATE 中的 SET 子句, 智能去除尾部逗号
- `<trim prefix=" " suffix=" " prefixOverrides=" " suffixOverrides=" ">`: 灵活的前缀/后缀处理

3.5 MyBatis 缓存机制 (补充细节) [p.22]

- > **一级缓存 (SqlSession 级别)**: 默认开启, 作用范围是同一个 SqlSession, 执行相同的 SELECT 语句时直接从缓存返回, 执行 INSERT/UPDATE/DELETE 后缓存自动清空
- > **二级缓存 (Mapper 级别)**: 需在 Mapper XML 中配置 `<cache/>` 标签开启, 跨 SqlSession 共享, 可集成第三方缓存 (Ehcache、Redis、Hazelcast 等)
- > 缓存执行顺序: 二级缓存 → 一级缓存 → 数据库查询

3.6 MyBatis 插件机制 (实现原理) [p.22]

- > 基于 Java 动态代理和责任链 (Chain of Responsibility) 模式实现
- > 可拦截的对象和方法: Executor (update, query, commit, rollback)、StatementHandler (prepare, parameterize, batch, update, query)、ParameterHandler (getParameterObject, setParameters)、ResultSetHandler (handleResultSets, handleOutputParameters)
- > 典型应用: 分页插件 (PageHelper)、性能监控、SQL 日志、读写分离

4 Hibernate 框架

4.1 Hibernate 简介 [p.29]

- > Hibernate 是 Java 生态中全自动 ORM 解决方案, 提供自然且类型安全的方式访问关系数据 [p.29]
- > 能快速编写复杂查询和处理结果, 也可以轻松将内存中修改同步到数据库 [p.29]
- > 遵循事务的 ACID 属性, 提供调优性能的能力 [p.29]
- > 与 Jakarta Persistence (JPA) 兼容, 可在支持 JPA 的任何环境中使用 [p.29]
- > 官网: <https://hibernate.org/> [p.31]

4.2 JPA 规范 (Java Persistence API) [p.30]

- > JPA 是 Java EE 的众多规范之一, 即 ORM 规范, 规定了对象持久化的 API 标准 [p.30]
- > JPA 本身不提供实现, 只提供规范接口; 广义上 JDBC、JdbcTemplate、MyBatis、Hibernate 都可以认为是 JPA 的实现 [p.30]
- > **JPA 规范的三大组成部分** [p.30]:
 - ORM 映射元数据 (Mapping Metadata): 包括 XML 和注解 (Annotation) 两种配置方式
 - 用于 Java 调用的 API 接口: EntityManager、EntityTransaction、Query 等
 - 面向对象的查询语言 JPQL (Java Persistence Query Language): 类似 SQL 但面向实体对象

4.3 Hibernate 核心特点 [p.32]

- > **(1) 完全的 ORM 能力**: 支持基于注解或 XML 定义实体与表的映射, 支持继承、多态、关联关系 (多对一 @ManyToOne、一对多 @OneToMany、多对多 @ManyToMany) 等复杂模型 [p.32]